Identifying LLMs from Their Responses

Zahid Rafeek

CMPSC 448: Midterm Project

11th October 2026

Assignment

Large Language Models (LLM) are capable of generating responses to questions that are similar in content and structure, making it pretty difficult to determine what model produced a particular response. This assignment attempts to determine whether or not neural networks can identify what the source LLM is from their responses.

The basic task is basically a classification problem. Given a question, a generated response, or both, the classifier predicts which of seven LLMs generated the response.

To do this, we focus on these three questions:

- RQ1: Can we identify which LLM generated a response? Use LLM_output as input, conduct classification to detect which LLM generated it. Report the performance of your CNN & RNN performance
- RQ2: Does the user's prompt help identify the LLM? Compare classifiers using: Input only; Output only; Input + Output. Questions to consider:
    - i) Can the LLM be predicted from the user's prompt even before seeing its response?
    - ii) Does including the input improve classification?
    - iii) If input-only classification works surprisingly well, why might this happen?
- RQ3: Do LLM fingerprints generalize across tasks or domains? Investigate whether a classifier trained on one type of task can identify LLMs on another type of task. For example: Training on problems like coding + mathematics + factual questions, but Testing on writing. Or Training on general question answering but Testing on coding. Questions you may consider:
    - i) Does performance decrease on unseen tasks?
    - ii) Are some models more consistently identifiable across domains?
    - iii) Is the classifier learning a general LLM fingerprint or task-specific patterns?

Two different Neural Networks Architectures were used and compared: a Convolutional Neural Network (CNN) and a Long Short-Term Memory Network (LSTM). The CNN is able to learn local patterns in the text, while the LSTM is able to process sequential data across a response.

Datasets

This project makes use of two different datasets for classification.

First Dataset

The first dataset is the "TXD-22 A Large-Scale Benchmark Dataset" which can be found here:
https://data.mendeley.com/datasets/prcjcggtjf/1.
This dataset consists of about 3000 question prompts that are the exact same between models covering 30 subjects, with roughly 100 academic questions per subject. These categories specifically aren't included in the dataset and were added when being filtered. Each question was asked to different LLM models.

For this project, I chose to classify 3 different models: ChatGPT, Claude, and Llama. Thus, there are 9000 in the filtered dataset.

The processed dataset is organized with the following fields:

Field	**Description**
prompt_id	Unique identifier for each question
prompt	Prompt submitted to the LLM
response	Generated response by the model
llm	Which model generated the response
category	What subject the response falls under

The LLM identity serves as the target label for supervised learning.
Although a category field was provided, it was not used for training or testing.

Second Dataset

The second dataset is the "LMSYS Judging LLM-as-a-judge with MT-Bench and Chatbot Arena" which can be found here: https://huggingface.co/datasets/lmsys/chatbot_arena_conversations.
This dataset describes more natural human prompts than the previous dataset and 2 different LLMs responses to that prompt. For this project, the data was filtered to only include the LLM models described above, of which only three existed in this dataset: ChatGPT, Claude, and Llama. This dataset is not categorized.

The processed dataset is organized with the following fields:

Field	**Description**
prompt	Prompt submitted to the LLM
response	Generated response by the model
llm	Which model generated the response

Data Preprocessing

Before training, all responses and prompts are filtered by LLM models into a Comma Separated Value file. Because all the data pertaining to one response has to be on one line, the prompts and responses are also processed to contain raw text data. For example, new lines are replaced with "\n", or tab spaces are replaced with "\t".

The text is converted into numerical representations using a vocabulary built from the training set. Words are tokenized by simply converting them to lowercase, mapped to integer IDs, and padded when necessary to create batches of consistent sequence length. Words not found in the training vocabulary are represented by an unknown-token identifier.

To avoid data leakage, all responses associated with the same question are kept within the same dataset split. The data itself is split into the following: 70% for training, 15% for validation, and 15% for testing when using the same dataset for training and testing.

Otherwise, a stratified split is used to break up one dataset into training and validation, and the other dataset is used for testing.

Neural Network Implementation and Training

Two neural network architectures were implemented using the PyTorch library.

Convolutional Neural Network (CNN)

The CNN begins with an embedding layer that converts token IDs into dense vectors. One or more convolutional layers process neighboring token vectors to identify local patterns. A nonlinear activation function and pooling layer summarize the extracted features. The resulting representation is passed to a fully connected classification layer that produces scores for the seven (or three depending on the dataset) LLM classes. This architecture is suitable for investigating whether recurring phrases, word combinations, punctuation patterns, or other local textual characteristics help distinguish the models.

Responses used for training are padded to ensure same sized inputs into the network

Long Short-Term Network (LSTM)

The LSTM also uses an embedding layer, but processes tokens sequentially. Its recurrent hidden state and gating mechanisms allow information from earlier tokens to influence the representation of later tokens. A sequence representation is passed to a classification layer to predict the source LLM.
This architecture is intended to capture sequential patterns and relationships that may extend beyond a small group of neighboring words.

Input Configurations

Each of these networks are evaluated using three different input methods:

1. Response-only: Uses only the LLM responses for categorization
2. Prompt-only: Uses the original prompt for categorization
3. Combined: Uses both the original prompt and generated response for categorization.

These configurations are used to investigate whether the response contains useful identifying patterns and whether the prompt provides additional predictive information.

The same label definitions and data partitioning strategy are used across input configurations to support fair comparisons.

Padding is ignored, as it can cause the hidden layers to train on padding and not the actual response itself.

Training Procedure

The models are trained using supervised learning. For each training iteration (epoch), the model receives a batch of encoded text and produces a score for each LLM class. A classification loss measures the difference between the predictions and the correct label.

For our loss function, we use Cross-Entropy Loss.
For our optimizer, we use Adam with a learning rate of 0.001.

The vocabulary and other preprocessing components are fitted only using the training data.
All model configurations were run with an epoch of 10.
For consistency, the random values were seeded to be 45.
The batch size for the data is 32.

To build the vocabulary for the models to use, a simple word frequency lookup list based on the training data provided was used.

Model Settings

For the CNN:

- Embedding dimension: 128
- Padding index: 0
- Convolution layer (1 Dimensional): 128 input, 128 output, kernel size 5
- Adaptive pooling

For the LSTM:

- Embedding size: 128
- Hidden layer size: 128

Results

Can we identify which LLM generated a response and does the user's prompt help identify the LLM?

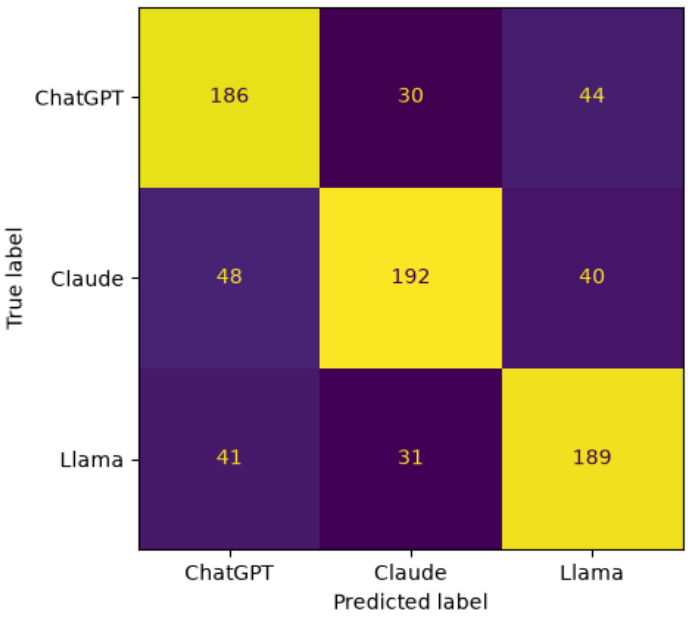
This question uses the LLM responses, the users' prompts, and a combination of both as the input for the Neural Networks. The training for this question will use both the LMSYS dataset and the TXD-22 dataset separately.

CNN - LLM Response

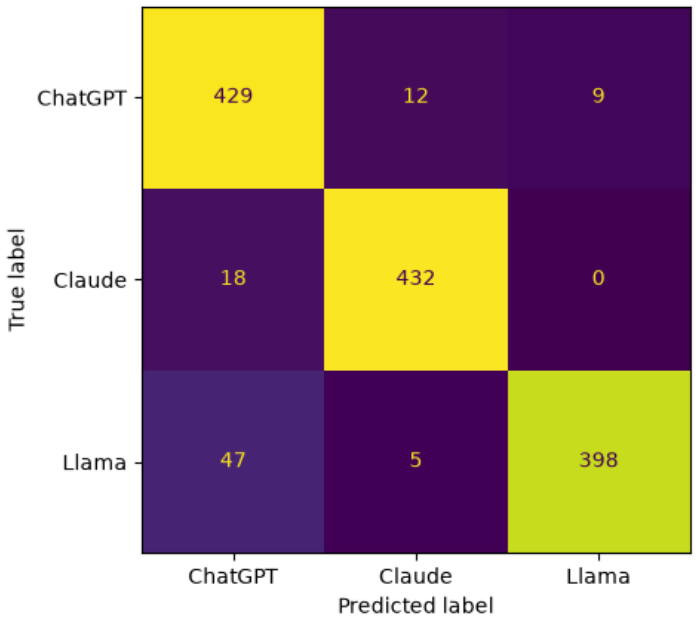
Evaluation Table - LMSYS			
Model	**Precision**	**Recall**	**F1-Score**
ChatGPT	0.68	0.72	0.70
Claude	0.76	0.69	0.72
Llama	0.69	0.72	0.71
Accuracy			**0.71**
Macro Avg.	**0.71**	**0.71**	**0.71**

Evaluation Table - TXD-22			
Model	**Precision**	**Recall**	**F1-Score**
ChatGPT	0.87	0.95	0.91
Claude	0.96	0.96	0.96
Llama	0.98	0.88	0.93
Accuracy			**0.93**
Macro Avg.	**0.94**	**0.93**	**0.93**

Confusion Matrix (LMSYS):



Confusion Matrix (TXD-22):

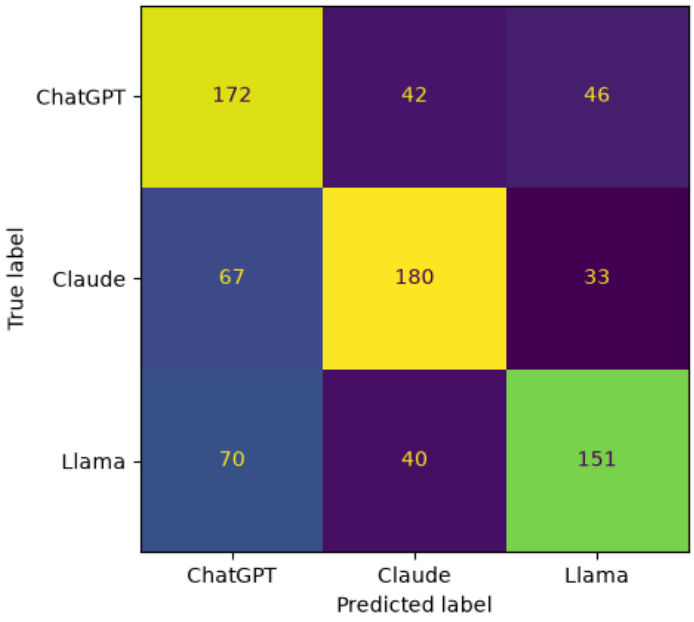


LSTM - LLM Response

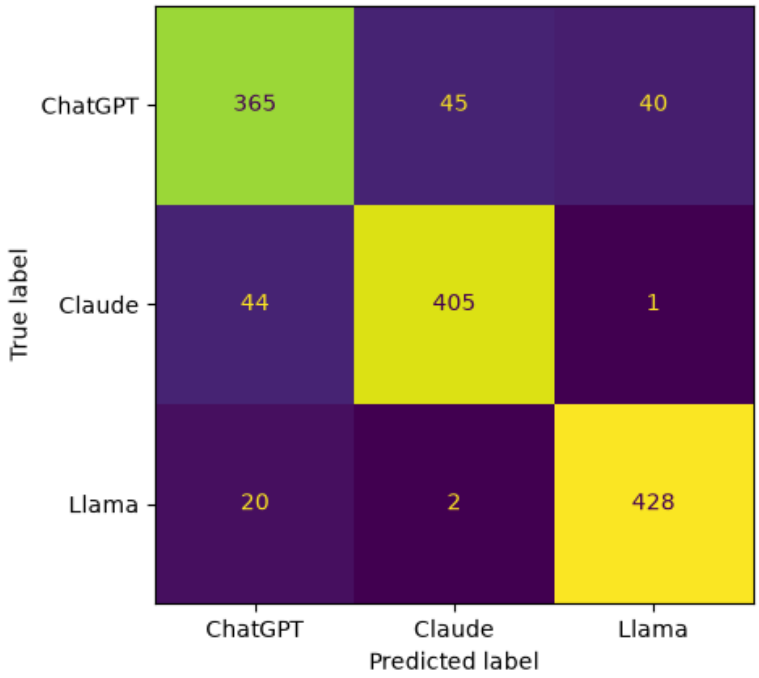
Evaluation Table - LMSYS			
Model	**Precision**	**Recall**	**F1-Score**
ChatGPT	0.56	0.66	0.60
Claude	0.69	0.64	0.66
Llama	0.66	0.58	0.62
Accuracy			**0.63**
Macro Avg.	**0.63**	**0.63**	**0.63**

Evaluation Table - TXD-22			
Model	**Precision**	**Recall**	**F1-Score**
ChatGPT	0.85	0.81	0.83
Claude	0.90	0.90	0.90
Llama	0.91	0.95	0.93
Accuracy			**0.89**
Macro Avg	**0.89**	**0.89**	**0.89**

Confusion Matrix (LMSYS):



Confusion Matrix (TXD-22):

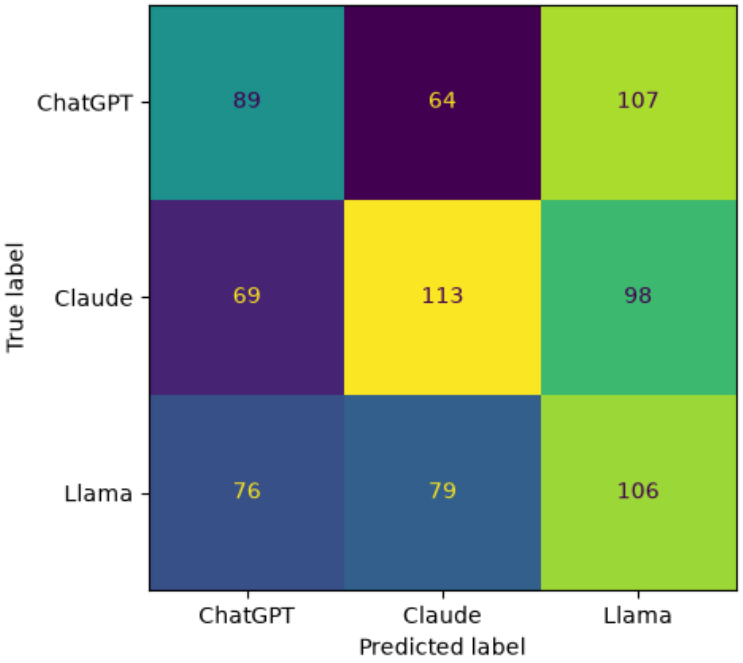


CNN - User Prompt

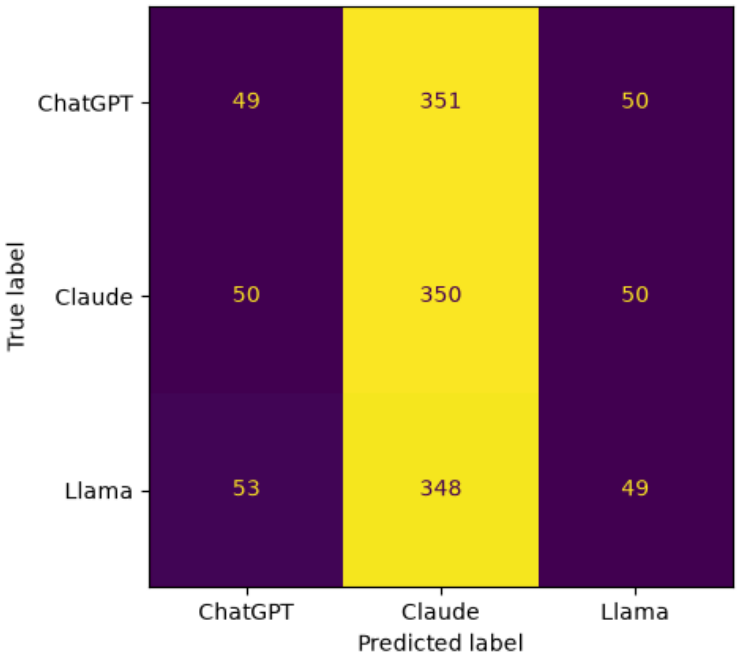
Evaluation Table - LMSYS			
Model	**Precision**	**Recall**	**F1-Score**
ChatGPT	0.38	0.34	0.36
Claude	0.44	0.40	0.42
Llama	0.34	0.41	0.37
Accuracy			**0.38**
Macro Avg.	**0.39**	**0.38**	**0.38**

Evaluation Table - TXD-22			
Model	**Precision**	**Recall**	**F1-Score**
ChatGPT	0.32	0.11	0.16
Claude	0.33	0.78	0.47
Llama	0.33	0.11	0.16
Accuracy			**0.33**
Macro Avg.	**0.33**	**0.33**	**0.26**

Confusion Matrix (LMSYS):



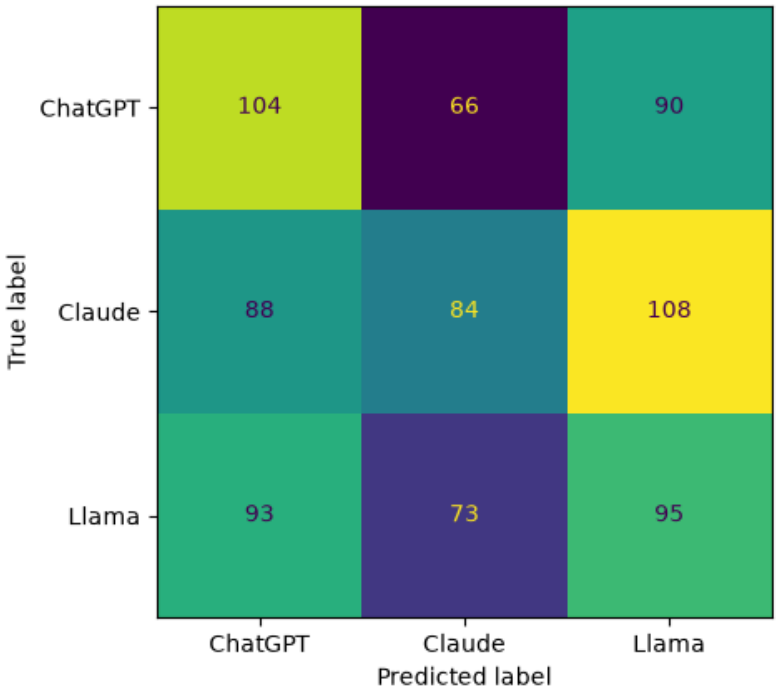
Confusion Matrix (TXD-22):



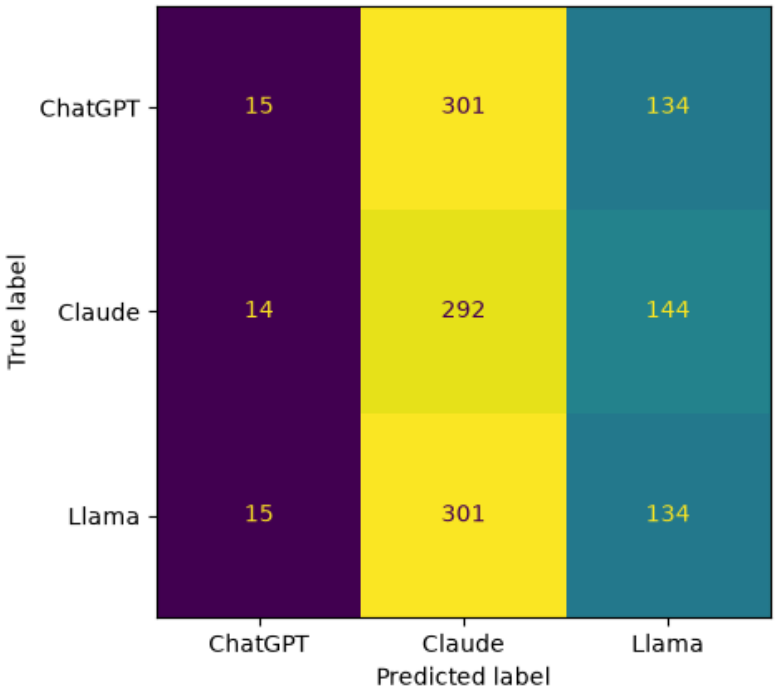
LSTM - User Prompt

Evaluation Table - LMSYS			
Model	**Precision**	**Recall**	**F1-Score**
ChatGPT	0.36	0.40	0.38
Claude	0.38	0.30	0.33
Llama	0.32	0.36	0.34
Accuracy			**0.35**
Macro Avg.	**0.36**	**0.35**	**0.35**

Evaluation Table - TXD-22			
Model	**Precision**	**Recall**	**F1-Score**
ChatGPT	0.34	0.03	0.06
Claude	0.33	0.65	0.43
Llama	0.33	0.30	0.31
Accuracy			**0.33**
Macro Avg.	**0.33**	**0.33**	**0.27**



Confusion Matrix (LMSYS):



CNN - Combination

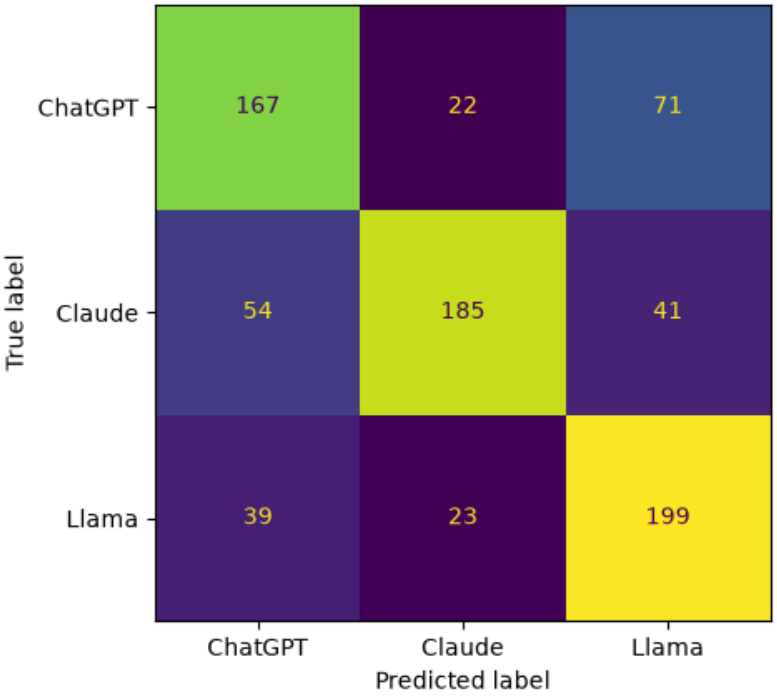
Evaluation Table - LMSYS

Model	Precision	Recall	F1-Score
ChatGPT	0.64	0.64	0.64
Claude	0.80	0.66	0.73
Llama	0.64	0.76	0.70
Accuracy			**0.69**
Macro Avg.	**0.70**	**0.69**	**0.69**

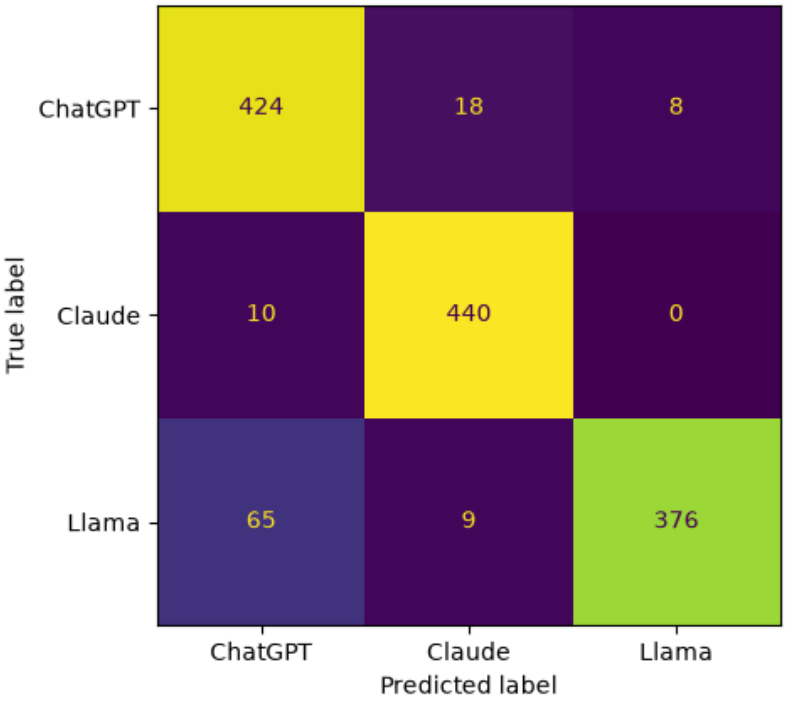
Evaluation Table - TXD-22

Model	Precision	Recall	F1-Score
ChatGPT	0.85	0.94	0.89
Claude	0.94	0.98	0.96
Llama	0.98	0.84	0.90
Accuracy			**0.92**
Macro Avg.	**0.92**	**0.92**	**0.92**

Confusion Matrix (LMSYS):



Confusion Matrix (TXD-22):



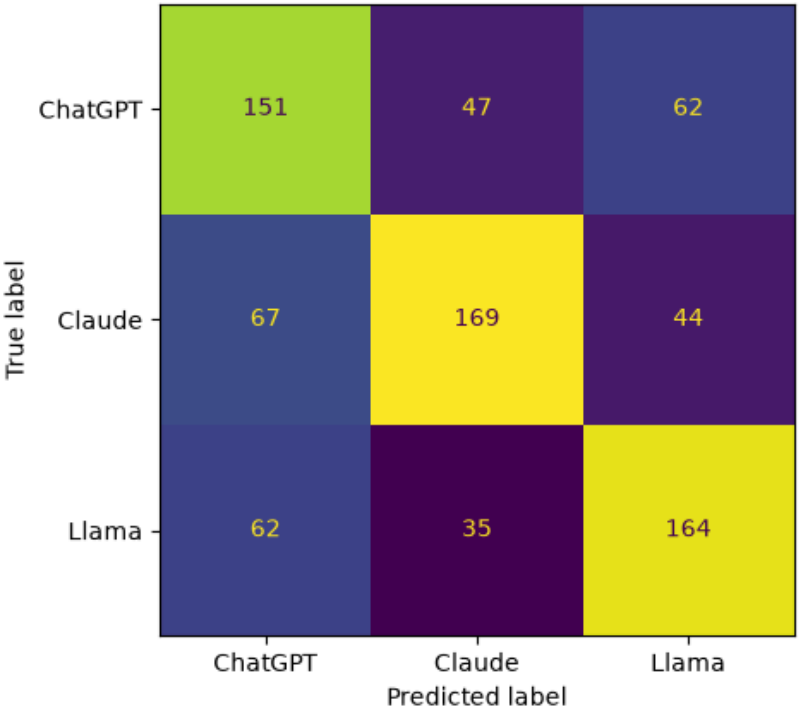
LSTM - Combination

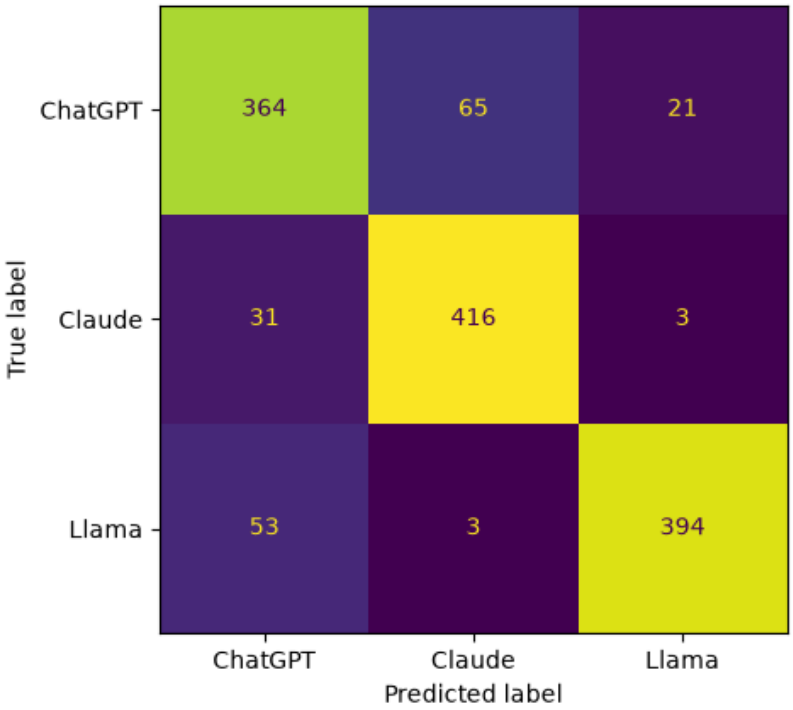
Evaluation Table - LMSYS

Model	Precision	Recall	F1-Score
ChatGPT	0.54	0.58	0.56
Claude	0.67	0.60	0.64
Llama	0.61	0.63	0.62
Accuracy			**0.60**
Macro Avg.	**0.61**	**0.60**	**0.60**

Evaluation Table - TXD-22

Model	Precision	Recall	F1-Score
ChatGPT	0.81	0.81	0.81
Claude	0.86	0.92	0.89
Llama	0.94	0.88	0.91
Accuracy			**0.87**
Macro Avg.	**0.87**	**0.87**	**0.87**





Results and Analysis

When using the LMSYS dataset, the prompt-only experiments achieved 38% accuracy with the CNN and 37% with the LSTM, modestly exceeding the approximately 37% majority-class baseline. However, this dataset does not have many overlaps of prompts between LLMs, which may cause the Neural Networks to exploit differences in the types of prompts associated with each model rather than identify model-specific characteristics when using only the prompt as the training data.This limitation also affects the interpretation of combined-input results, since prompt content may provide dataset-specific predictive information.

When using the TXD-22 dataset, the prompt-only classification achieves approximately 35% accuracy, close to the 33% random-chance baseline. This dataset uses the same prompt for each model. Combining prompts with responses provides only a small improvement for the CNN and no measurable improvement for the LSTM. In contrast, prompt-only classification performs substantially better on LMSYS, where prompts are rarely shared between models. This difference suggests that prompt-label correlations may contribute to LMSYS classification performance. Therefore, prompt-only results should be interpreted cautiously, since they may reflect dataset-specific prompt distributions rather than generalizable model-identification characteristics.

The TXD-22 results provide evidence that the prompt alone is insufficient for reliable identification in this experimental setup. Both architectures achieve only 33% accuracy, with most predictions concentrated on a small number of classes.
This contrasts with LMSYS, where prompt-only accuracy was 41% for the CNN and 37% for the LSTM. Because LMSYS prompts are rarely shared across models, its higher scores may reflect correlations between prompt content and model labels. TXD-22's shared-prompt design reduces this particular factor.

The comparison with LMSYS suggests that dataset construction may explain some of the difference. When prompts differ across models, prompt content can become correlated with the model label. When the same prompts are used across models, as in TXD-22, that source of predictive information is substantially reduced.
However, this comparison does not prove that prompt overlap is the only cause of the difference. The datasets may also differ in model versions. For instance, one dataset may be using ChatGPT-4o, whilst the other might be using ChatGPT-5, but both are labeled under “ChatGPT”.

Do LLM fingerprints generalize across tasks or domains?

This question was done using two cross-dataset experiments:

- Experiment 1: Train on the LMSYS dataset and test on the TXD-22 dataset.
- Experiment 2: Train on the TXD-22 dataset and test on the LMSYS dataset.

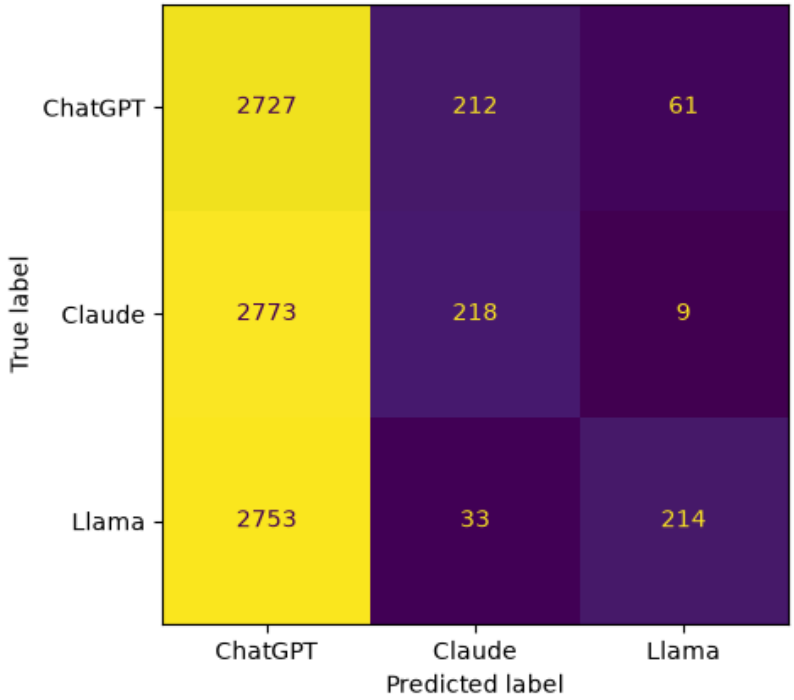
The TXD-22 dataset prompts are entirely academic questions, whereas the LMSYS dataset uses more natural human prompts with only about 50 of the prompts total being academic related.

Experiment 1: CNN

Evaluation Table

Model	**Precision**	**Recall**	**F1-Score**
ChatGPT	0.33	0.91	0.48
Claude	0.47	0.07	0.13
Llama	0.75	0.07	0.13
Accuracy			**0.35**
Macro Average	**0.52**	**0.35**	**0.25**

Confusion Matrix:

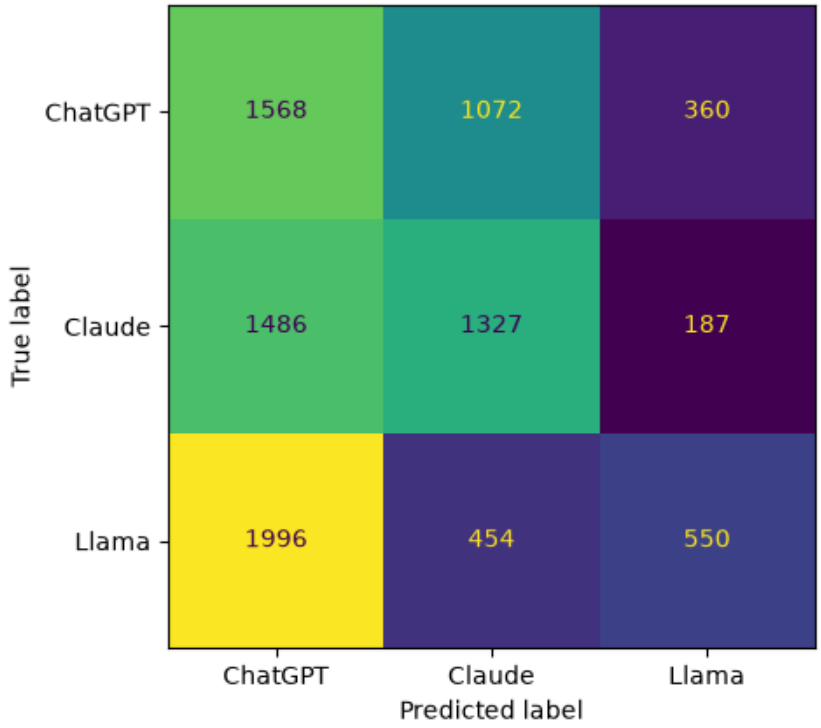


Experiment 1: LSTM

Evaluation Table

Model	**Precision**	**Recall**	**F1-Score**
ChatGPT	0.31	0.52	0.39
Claude	0.47	0.45	0.45
Llama	0.50	0.18	0.27
Accuracy			**0.38**
Macro Average	**0.43**	**0.38**	**0.37**

Confusion Matrix:



Experiment 1: Results

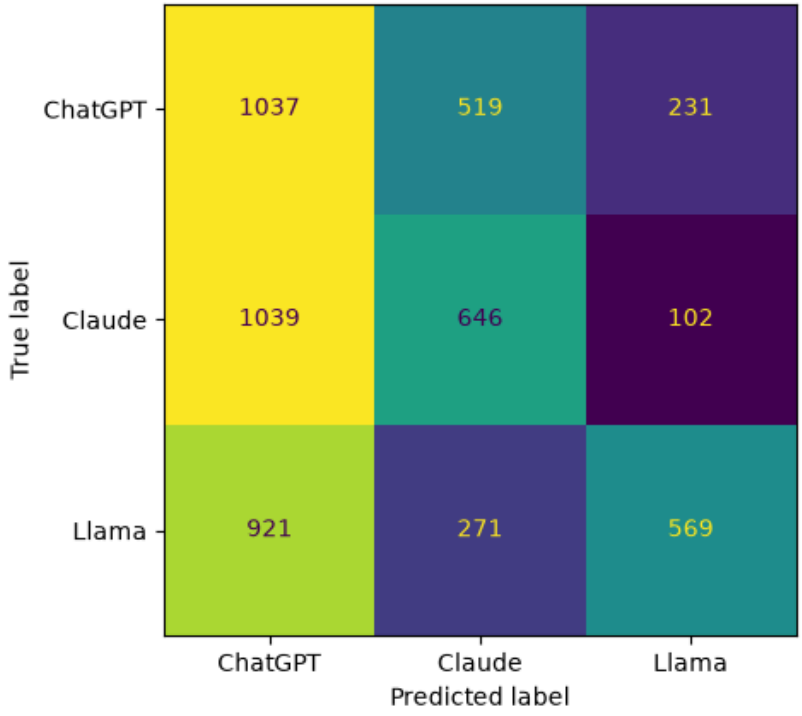
The CNN achieved 35% accuracy and a macro F1-score of 0.25. Its predictions were heavily biased toward ChatGPT: it correctly identified 2,727 of 3,000 ChatGPT examples, but correctly identified only 218 Claude examples and 214 Llama examples. The LSTM achieved 38% accuracy and a macro F1-score of 0.37. Its predictions were more distributed across the classes, although Llama recall remained low at 0.18. Neither model demonstrated strong cross-dataset identification.

Experiment 2: CNN

Evaluation Table

Model	**Precision**	**Recall**	**F1-Score**
ChatGPT	0.35	0.58	0.43
Claude	0.45	0.36	0.40
Llama	0.63	0.32	0.43
Accuracy			**0.42**
Macro Average	**0.48**	**0.42**	**0.42**

Confusion Matrix:

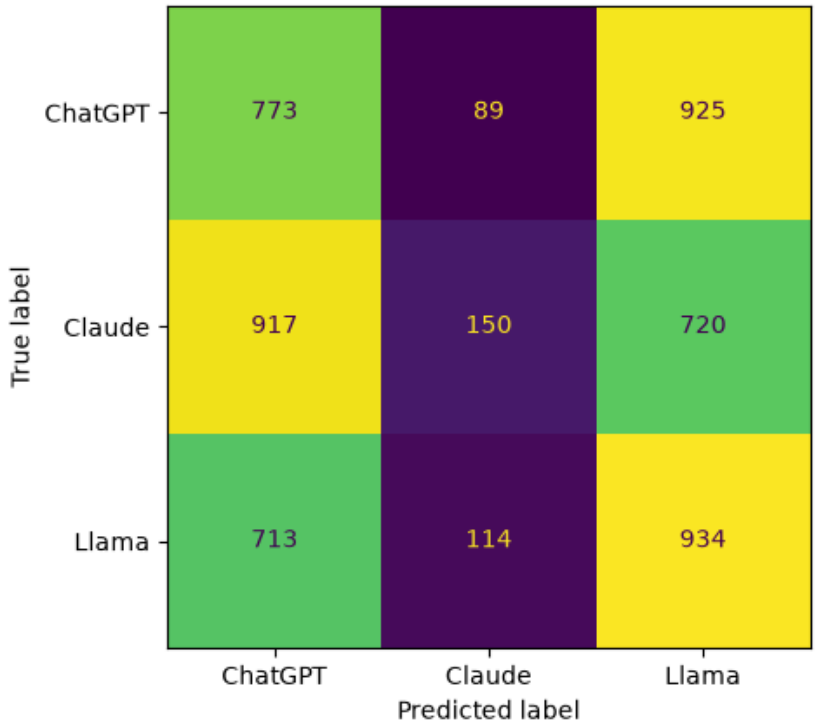


Experiment 2: LSTM

Evaluation Table

Model	**Precision**	**Recall**	**F1-Score**
ChatGPT	0.32	0.43	0.37
Claude	0.42	0.08	0.14
Llama	0.36	0.53	0.43
Accuracy			**0.35**
Macro Average	**0.37**	**0.35**	**0.31**

Confusion Matrix:



Experiment 2: Results

The CNN achieved 42% accuracy and a macro F1-score of 0.42. The LSTM achieved 35% accuracy and a macro F1-score of 0.31. The CNN performed better overall, particularly for Claude, whose recall was 0.36 compared with just 0.08 for the LSTM. However, neither model consistently identified all three classes well. The fact that the CNN performs better in one direction and the LSTM in the other suggests that architecture alone does not explain cross-dataset performance. Differences in the datasets and their response distributions may play an important role.

Lessons Learnt

This project demonstrated that neural networks can learn patterns associated with the source of an LLM response. Within TXD-22, the CNN achieved 93% accuracy and a macro F1-score of 0.93, while the LSTM achieved 86% accuracy and a macro F1-score of 0.86. However, performance varied by model, showing the importance of examining per-class precision, recall, and F1-score instead of relying on overall accuracy alone.

The model identification experiments showed that prompts alone were not reliable indicators of model identity in TXD-22, where all models received the same prompts. Combining prompts with responses did not improve the reported results over using responses alone. This suggests that the generated responses contained more useful information for classification than the prompts in these experiments, though further testing would be needed to determine why.

The cross-domain experiments highlighted the difficulty of transferring learned patterns between datasets. Accuracy declined to between 35% and 42% when a model was trained on one dataset and tested on the other. These results suggest that a classifier can perform well on data from the same source but fail to generalize reliably to data collected under different conditions.

The project also highlighted the importance of careful dataset splitting and consistent preprocessing. Keeping responses to the same prompt within a single split helps reduce data leakage, while fitting the vocabulary on training data prevents test data from influencing preprocessing. Finally, comparing multiple architectures and examining confusion matrices revealed differences that overall accuracy alone would not have shown.

Future experiments could investigate whether prompt composition, response length, formatting, or model versions contribute to the cross-dataset performance decline.